

Item 6 — Tecnologias Sugeridas

Prompt

Você é um arquiteto de software trabalhando na Phase D (Technology Architecture) do TOGAF ADM.

Contexto:

- Domínio: app web de descoberta e tracking de video games (estilo Letterboxd para jogos).
- Arquitetura (item 3): 3-tier — SPA frontend, API REST stateless backend, banco relacional + cache, integração read-only com a RAWG API.
- Autenticação JWT, autorização RBAC; entidades e endpoints já definidos (itens 4-5).
- Perfil do time: desenvolvedores full-stack; prioridade em produtividade, ecossistema maduro e baixo custo de operação para um MVP acadêmico que pode evoluir para produção.

Tarefa:

Sugira a stack de tecnologias, organizada por camada/responsabilidade.

1. Para cada camada (frontend, backend, banco de dados, cache, autenticação, integração externa, infraestrutura/deploy, observabilidade), indique a tecnologia recomendada e 1 frase justificando a escolha.
2. Quando fizer sentido, ofereça uma alternativa equivalente (ex: "PostgreSQL ou MySQL").
3. Liste as decisões transversais: linguagem principal, gerenciamento de dependências, testes, CI/CD e containerização.

Restrições:

- Pode ser genérico onde não importa ("banco relacional"), mas seja específico onde a escolha tem impacto arquitetural (ex: cache, ORM, estratégia de auth).
- Não introduzir tecnologias que exijam integrações externas além da RAWG API.
- Justificar escolhas com base em produtividade, maturidade do ecossistema e adequação ao domínio, não modismo.

Formato de saída:

- Tabela Markdown: Camada | Tecnologia recomendada | Alternativa | Justificativa.
- Seção "Decisões transversais": bullets.

Output (rascunho, validar e refinar com a ferramenta)

Substitua pelas tecnologias que você realmente domina/usa profissionalmente — a atividade valoriza a experiência prática refletida na escolha.

Stack por camada

#	Camada	Tecnologia recomendada	Alternativa	Justificativa
1	Frontend (SPA)	React + TypeScript	Vue / Angular	Ecossistema maduro, tipagem estática, grande oferta de bibliotecas de UI e estado.
2	Estilização/UI	Tailwind CSS + componentes shadcn/ui	Material UI	Prototipagem rápida e visual consistente sem CSS solto.
3	Backend (API REST)	Node.js + NestJS (TypeScript)	Spring Boot (Java) / Django REST (Python)	Mesma linguagem do front, arquitetura modular opinativa, suporte first-class a DI, guards (RBAC) e validação.
4	ORM / acesso a dados	Prisma	TypeORM / Hibernate	Migrations versionadas, tipagem ponta a ponta, produtividade alta.
5	Banco de dados	PostgreSQL	MySQL	Relacional robusto, integridade referencial, bom para consultas agregadas de estatísticas.
6	Cache	Redis	Memcached	Cacheia respostas da RAWG e dados quentes, reduz latência e chamadas externas; suporta TTL e rate-limit.
7	Autenticação	JWT (access + refresh) com bcrypt para hash	Sessions + Redis	Stateless, escala horizontal, integra direto com guards RBAC do backend.
8	Integração externa	Cliente HTTP da RAWG com cache-aside no Redis	—	Encapsula chave de API, normaliza payload e respeita rate limit.
9	Infra / Deploy	Docker + Docker Compose; deploy em Render/Railway/Fly.io	Kubernetes (se escalar)	Containerização reprodutível; PaaS de baixo custo para MVP acadêmico.
10	Observabilidade	Logs estruturados (pino) + health checks	OpenTelemetry + Grafana	Diagnóstico básico no MVP, evolutível para tracing distribuído.

Decisões transversais

- **Linguagem principal:** TypeScript ponta a ponta (front + back), reduzindo troca de contexto e permitindo compartilhar tipos/DTOs.
- **Gerenciamento de dependências:** pnpm (monorepo) ou npm workspaces separando frontend e backend .
- **Testes:** unitários e de integração com Jest; testes de contrato da API com Supertest.
- **CI/CD:** GitHub Actions — lint + test + build em PR, deploy automático na branch principal.
- **Containerização:** Dockerfile por serviço + Docker Compose para subir API + Postgres + Redis localmente.
- **Configuração/segredos:** variáveis de ambiente (.env) com a chave da RAWG fora do versionamento.

Critérios de aceite

- ☐ Tecnologia sugerida para cada camada da arquitetura do item 3
- ☐ Específico onde a escolha tem impacto arquitetural (cache, ORM, auth), genérico onde não importa
- ☐ Alternativa equivalente oferecida onde fizer sentido

- ☐ Justificativas baseadas em produtividade/maturidade/adequação, não modismo
 - ☐ Decisões transversais cobrindo linguagem, dependências, testes, CI/CD e containerização
 - ☐ Nenhuma tecnologia exige integração externa além da RAWG API
-

Ferramenta utilizada: Claude (Anthropic), modelo Opus 4.8.